

Week 01: Lean as a functional language

Functional Programming in Lean

Learning goals

- ▶ Read a Lean type and a function definition
- ▶ Use `#eval` and `#check` for feedback
- ▶ Explain why a pure expression can be replaced by its value

Central idea

A function maps inputs to outputs. Its type is a compact contract.

Lean example

```
def double (n : Nat) : Nat := n + n  
#eval double 7  
#check double
```

What to notice

- ▶ Lean checks definitions as the file changes
- ▶ A type error identifies a mismatch between expected and actual types
- ▶ `#eval` runs code; `#check` reports a type

Live coding

1. Complete `double` in `Templates/Week01.lean`
2. Write `larger` using `if`
3. Predict both evaluations before running them

Pause and predict

What is the type of `fun n : Nat => n + 1`?

Exercise connection

Open `exercises/Week01.md` after the lecture.

The complete example is `Course/Week01.lean`; the fill-in version is `Templates/Week01.lean`.